Solution: Rotate Image

Problem Statement

Given an `n x n` 2D matrix, modify a square matrix by `rotating` it `90 degrees` in a `clockwise` direction.

Note: This rotation should be done `in-place`, meaning the transformation must occur within the original matrix without using any additional space for another matrix.

Examples

- **Example 1**:
    - **Input**: matrix =

```
[[1,2],
 [3,4]]
```

- **Expected Output**:

```
[[3,1],
 [4,2]]
```

- **Justification**: After rotating the 2x2 matrix 90 degrees to the right, the element at the top left (1) moves to the top right, the top right (2) moves to the bottom right, the bottom right (4) moves to the bottom left, and the bottom left (3) moves to the top left.

- **Example 2**:
    - **Input**: matrix =

```
[[5,1,9],
 [2,4,8],
 [13,3,6]]
```

- **Expected Output**:

```
[[13,2,5],
 [3,4,1],
 [6,8,9]]
```

- **Justification**: Rotating the 3x3 matrix 90 degrees to the right repositions the first row to the last column, the second row to the middle column, and the third row to the first column.

- **Example 3**:
    - **Input**: matrix =

```
[[10,11,12,13],
 [14,15,16,17],
 [18,19,20,21],
 [22,23,24,25]]
```

- **Expected Output**:

```
[[22,18,14,10],
 [23,19,15,11],
 [24,20,16,12],
 [25,21,17,13]]
```

- **Justification**: The matrix is rotated by 90 degrees in the clockwise direction.

Solution

To solve this problem, we'll employ a two-step approach: first, transpose the matrix, and then reverse each row. Transposing the matrix swaps the row and column indices for each element, effectively rotating the matrix by 90 degrees but in the wrong direction. Reversing each row afterward aligns the matrix correctly for the desired 90-degree rotation to the right.

This method is effective because it directly manipulates the matrix in place, adhering to the in-place requirement, and it systematically rearranges the elements to achieve the rotation without needing additional storage.

Step-by-step Algorithm

1. **Transpose the Matrix**:
    - Iterate over the matrix with two nested loops, where the outer loop variable `i` runs from 0 to `n-1` (inclusive) and the inner loop variable `j` runs from `i` to `n-1` (inclusive).
    - For each pair `(i, j)`, swap the elements at positions `[i][j]` and `[j][i]`. This effectively changes rows into columns, transposing the matrix.
2. **Reverse Each Row**:
    - After the matrix is transposed, iterate over each row of the matrix with a single loop where the loop variable `i` runs from 0 to `n-1` (inclusive).
    - For each row `i`, reverse the elements in the row. To do this, use a second loop where you swap elements from the start and end of the row moving towards the center. The loop variable `j` runs from 0 to `(n/2)-1` (inclusive), and for each iteration, swap the elements at positions `[i][j]` and `[i][n-1-j]`.

Algorithm Walkthrough

Let's consider the input:

```
[[10, 11, 12, 13],
 [14, 15, 16, 17],
 [18, 19, 20, 21],
 [22, 23, 24, 25]]
```

1. **Transpose the Matrix**:
    - Swap (11, 14), (12, 18), (13, 22) for the first row.
    - Swap (16, 19), (17, 23) for the second row.
    - Swap (21, 24) for the third row.
    - The matrix after transposition:

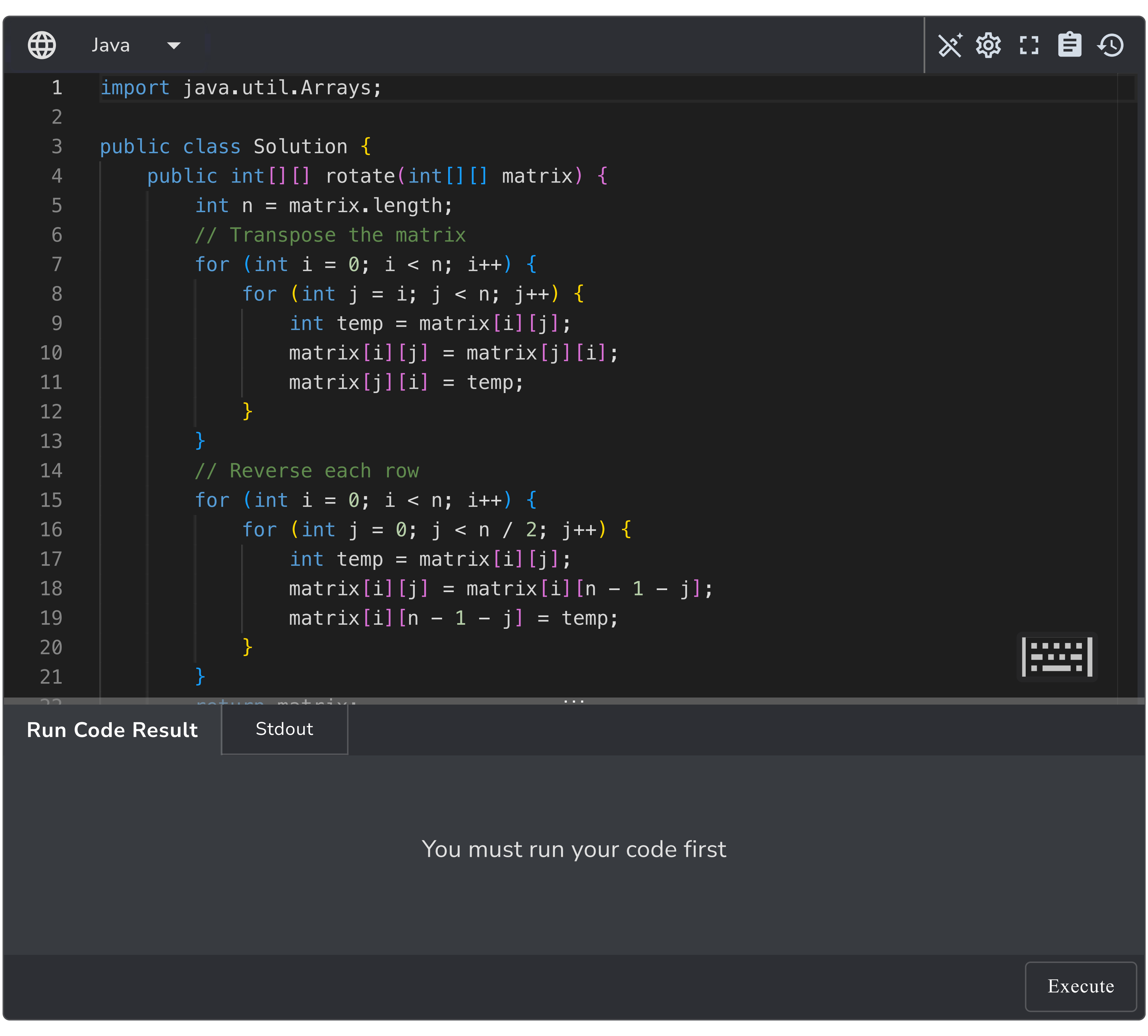
    ```
    [[10, 14, 18, 22],
     [11, 15, 19, 23],
     [12, 16, 20, 24],
     [13, 17, 21, 25]]
    ```

2. **Reverse Each Row**:
    - Reverse the first row: `[22, 18, 14, 10]`
    - Reverse the second row: `[23, 19, 15, 11]`
    - Reverse the third row: `[24, 20, 16, 12]`
    - Reverse the fourth row: `[25, 21, 17, 13]`
    - The final rotated matrix:

    ```
    [[22, 18, 14, 10],
     [23, 19, 15, 11],
     [24, 20, 16, 12],
     [25, 21, 17, 13]]
    ```

Each step transforms the matrix closer to the final rotated form. By first transposing the matrix, we align the rows and columns to their new orientations, and by then reversing each row, we correct the order of elements to match the 90-degree rotation to the right.

Code



```java
import java.util.Arrays;

public class Solution {
    public int[][] rotate(int[][] matrix) {
        int n = matrix.length;
        // Transpose the matrix
        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }
        // Reverse each row
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n / 2; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[i][n - 1 - j];
                matrix[i][n - 1 - j] = temp;
            }
        }
```

Complexity Analysis

Time Complexity

- $O(n^2)$: The algorithm iterates over each element of the matrix twice. First, during the transposition step, each element is visited once. Second, when reversing the rows, each element is again accessed once. Since the matrix is of size `n x n`, the total time complexity is $O(n^2)$.

Space Complexity

- $O(1)$: The rotation is performed in-place, which means no additional storage is required beyond temporary variables that are used for swapping elements. This results in a constant space complexity.

← Previous
Rotate Image (Medium)
Next →
Combinations (Medium)
☐ Mark as Completed